

TUGAS ALGORITMA STRUKTUR DATA PERTEMUAN 11: GRAPH

Nama: Firly Ramadhani Tubagus

NIM: J0403251115

Kelas: TPL A1

1. Praktikum 1: Membuat Adjacency Matrix

Code:

```
1 #=====
2 #Nama: Firly Ramadhani T
3 #NIM: J0403251115
4 #Kelas: A1
5 #=====
6 def createGraph(V,edges):
7     mat = [[0 for _ in range(V)] for _ in range(V)]
8
9     #Menambahkan setiap edge ke adjacency matrix
10    for it in edges:
11        u = it[0]
12        v = it[1]
13        mat[u][v] = 1
14
15        #karena graphnya tidak memiliki arah jadi setiap bertemu dijadikan 1
16        mat[v][u]=1
17    return mat
18
19 if __name__=="__main__":
20     V = 4
21
22     #list edgesnya
23     edges = [[0,1],[0,2],[1,2],[2,3]]
24
25     #membuat graphnya menggunakan edges
26     mat = createGraph(V,edges)
27
28     #memanggil
29     print("Adjacency Matrix:")
30     for i in range(V):
31         for j in range(V):
32             print(mat[i][j],end=" ")
33         print()
```

Output:

```
PS E:\IPB\TPL\SEM2\ALGORITMA\ASD_J0403251115_I
TMA\ASD_J0403251115_I
Adjacency Matrix:
0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0
```

2. Praktikum 2: Membuat Adjacency List

Code:

```
6 def createGraph(edges):
7     adj={}
8     #menambahkan edge ke adjacency listnya
9     for it in edges:
10        u=it[0]
11        v=it[1]
12
13        #kalau node belum ada
14        if u not in adj:
15            adj[u]=[]
16
17        if v not in adj:
18            adj[v]=[]
19
20        #tambahkan edge
21        adj[u].append(v)
22        #karena graphnya tidak berarah
23        adj[v].append(u)
24    return adj
25
26 if __name__=="__main__":
27     #list edges nya
28     edges = [[["A","B"],["A","C"],["B","D"],["C","D"]]]
29     #membuat graph menggunakan edgesnya
30     adj = createGraph(edges)
31     #memanggil
32     print("Adjacency List:")
33     for node in adj:
34         print(f"{node}: ", end=" ")
35         for j in adj[node]:
36             print(j, end=" ")
37         print()
```

Output:

```
PS E:\IPB\TPL\SEM2\ALGORITMA\ASD_J0403251115_FIRLYR
TMA\ASD_J0403251115_FIRLYR
Adjacency List:
A: B C
B: A D
C: A D
D: B C
PS E:\IPB\TPL\SEM2\ALGORITMA\ASD_J0403251115_FIRLYR
```

3. Praktikum 3: Konversi Matrix ke List

Code:

```
6 def createGraph(edges):
7     adj = {}
8     #looping tiap baris matrix
9     for i in range(len(edges)):
10
11         #list kosong untuk node i
12         adj[i] = []
13
14         for j in range(len(edges[i])):
15
16             if edges[i][j] == 1:
17                 adj[i].append(j)
18
19     return adj
20
21 if __name__ == "__main__":
22
23     matrix = [
24         [0,1,1,0],
25         [1,0,1,0],
26         [1,1,0,1],
27         [0,0,1,0]
28     ]
29     #membuat graphnya
30     adj = createGraph(matrix)
31     print("Adjacency List:")
32     for node in adj:
33         print(f"{node}: ", end=" ")
34         for j in adj[node]:
35             print(j, end=" ")
36         print()
```

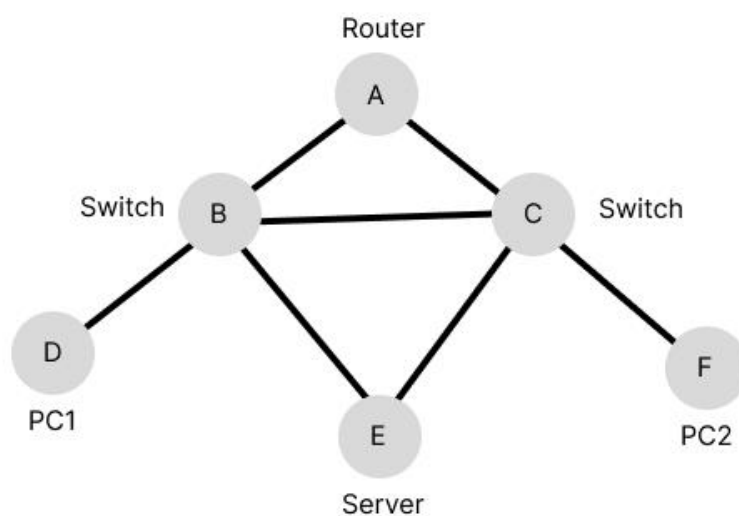
Output:

```
PS E:\IPB\TPL\SEM2\A
TMA\ASD_J0403251115_
Adjacency List:
0: 1 2
1: 0 2
2: 0 1 3
3: 2
PS E:\IPB\TPL\SEM2\A
```

4. Praktikum 4: Studi Kasus Dunia Nyata

Vertex = {A,B,C,D,E,F} (6 vertex)

Edge = {(A,B),(A,C),(B,C),(B,D),(B,E),(C,E),(C,F)} (7 edges)



Vertex = A → Router; B → Switch; C → Switch; D → PC1; E → Server; F → PC2

Edge = Koneksi Jaringan

- Implementasi dalam Python (Output)

1. Adjacency list

```
PS E:\IPB\TPL\SEM2\ALGORITMA\ASD_J0403251
TMA\ASD_J0403251115_FIRLYRAMADHANITUBAGUS
=== ADJACENCY LIST ===
A (Router) : B C
B (Switch) : A C D E
C (Switch) : A B E F
D (PC1) : B
E (Server) : B C
F (PC2) : C
```

2. Adjacency Matrix

```
=== ADJACENCY MATRIX ===
  A B C D E F
A 0 1 1 0 0 0
B 1 0 1 1 1 0
C 1 1 0 0 1 1
D 0 1 0 0 0 0
E 0 1 1 0 0 0
F 0 0 1 0 0 0
```

3. Nama Node

```
=== DAFTAR NODE ===
A = Router
B = Switch
C = Switch
D = PC1
E = Server
F = PC2
```

4. Hubungan antar Node

```
=== DAFTAR EDGE ===
A (Router) <-> B (Switch)
A (Router) <-> C (Switch)
B (Switch) <-> C (Switch)
B (Switch) <-> D (PC1)
B (Switch) <-> E (Server)
C (Switch) <-> E (Server)
C (Switch) <-> F (PC2)

Jumlah Vertex : 6
Jumlah Edge   : 7
```

- Analisis Singkat

Graph merepresentasikan topologi jaringan komputer sederhana. Setiap perangkat jaringan dianggap sebagai node (vertex), sedangkan hubungan atau koneksi antar perangkat dianggap sebagai edge. Pada graph ini terdapat router,

switch, server, dan client yang saling terhubung untuk membentuk jaringan komputer.

1. Jenis Graph (Directed/Undirected)

Pada studi kasus jaringan komputer ini menggunakan jenis Undirected Graph karena merupakan hubungan yang bersifat dua arah antar vertex/node. Seperti ketika Router A terhubung ke Switch B, maka Switch B terhubung juga ke Router A

2. Alasan memilih edge

Edge dipilih untuk merepresentasikan hubungan atau koneksi antar perangkat jaringan komputer (router, switch, server, juga pc). Pada graph ini:

Node A sebagai Router

Node B sebagai Switch

Node C sebagai Switch

Node D dan F sebagai PC

Node E sebagai Server

Setiap edge menunjukkan jalur komunikasi antar perangkat, seperti

A – B → Router terhubung ke Switch

B – D → Switch terhubung ke PC

C – E → Switch terhubung ke Server

3. Representasi yang lebih cocok

Pada studi kasus ini, representasi yang cocok adalah Adjacency List karena jumlah edge yang ada tidak terlalu banyak, lebih hemat memori dibandingkan adjacency matrix, dan lebih mudah membaca node yang saling terhubung

4. Graph termasuk Dense atau Sparse

Graph ini termasuk ke dalam Sparse Graph karena jumlah edge lebih sedikit dibandingkan jumlah maksimum edge yang mungkin. Karena jumlah vertex/node nya 6 sedangkan jumlah edgenya 7, jumlah maksimum edge yang memungkinkan adalah 15 tetapi graph ini hanya memiliki 7 edge, karena itu graph ini termasuk ke dalam sparse graph.

- Kesimpulan

Berdasarkan hasil studi kasus saya, graph dapat digunakan untuk merepresentasikan hubungan antar perangkat dalam jaringan komputer. Representasi adjacency list dan matrix membantu untuk memvisualisasikan koneksi antar node. Graph yang digunakan termasuk ke dalam undirected graph dan sparse graph karena jumlah edge yang lebih sedikit dibandingkan kemungkinan maksimum edge nya.